

FINAL PROJECT STRUKTUR DATA 2026

Final Project ini dikerjakan secara berkelompok. Setiap kelompok terdiri dari 5 orang sesuai pembagian kelompok yang telah ditentukan.

Pembagian kelompok dapat dilihat pada link berikut:

<https://docs.google.com/spreadsheets/d/1Q2IKkMeRNFLN1UMb-fZdLvuZV1aKq0ak8PEejKzD3Tg/edit?gid=362365178#gid=362365178>

Tujuan Final Project

Pada final project ini, mahasiswa diminta membuat aplikasi berbasis Java yang menggabungkan konsep **Tree** dan **Graph** untuk menyelesaikan suatu permasalahan nyata atau semi-nyata.

Project tidak hanya dinilai dari apakah program berjalan, tetapi juga dari kemampuan kelompok dalam:

1. Memilih struktur data yang tepat.
2. Menjelaskan alasan pemilihan Tree dan Graph.
3. Menerapkan algoritma dengan benar.
4. Menganalisis kompleksitas.
5. Menangani edge case.
6. Menunjukkan pemikiran kritis dan kreatif dalam menyelesaikan masalah.

Ketentuan Umum Project

Setiap kelompok wajib membuat aplikasi Java dengan ketentuan berikut:

1. Menggunakan minimal **1 struktur data Tree**.
2. Menggunakan minimal **1 struktur data Graph**.
3. Menggunakan minimal **2 algoritma Graph**.
4. Menggunakan dataset buatan sendiri.
5. Dataset minimal terdiri dari:
 - o 25 node/vertex.

- o 40 edge/relasi.
 - o Minimal 5 atribut tambahan sesuai topik.
6. Program wajib memiliki fitur:
- o Insert data.
 - o Search data.
 - o Update atau delete data.
 - o Menampilkan struktur graph.
 - o Menjalankan minimal 2 algoritma graph.
 - o Menjalankan minimal 1 operasi tree.
7. Tidak boleh menggunakan library graph siap pakai untuk algoritma utama.
8. Tidak boleh hanya membuat CRUD biasa.
9. Tidak boleh membuat topik di luar opsi yang tersedia.
10. Setiap kelompok wajib menjelaskan alasan pemilihan struktur data dan algoritma.

Struktur Data yang Boleh Digunakan

Tree

Kelompok boleh menggunakan salah satu atau lebih dari struktur berikut:

1. AVL Tree
2. Binary Search Tree
3. Heap / Priority Queue
4. Trie
5. Tree folder / hierarchical tree

Graph

Kelompok boleh menggunakan salah satu atau lebih dari representasi berikut:

1. Adjacency List
2. Adjacency Matrix
3. Weighted Graph
4. Directed Graph
5. Undirected Graph

Algoritma Graph

Kelompok wajib menggunakan minimal 2 dari algoritma berikut:

1. BFS
2. DFS
3. Dijkstra

4. A*
5. Minimum Spanning Tree
6. Topological Sort
7. Cycle Detection
8. Connected Component

Opsi Topik Final Project

Mahasiswa bebas memilih salah satu opsi berikut. Topik di luar daftar ini tidak diperbolehkan.

Opsi 1 — Smart Campus Route Planner

Membuat sistem pencarian rute antar lokasi di kampus, misalnya dari gerbang ke gedung, dari gedung kuliah ke kantin, atau dari parkir ke laboratorium.

Graph:

- Node merepresentasikan lokasi kampus.
- Edge merepresentasikan jalan antar lokasi.
- Edge wajib memiliki bobot, misalnya jarak, waktu tempuh, atau tingkat keramaian.
- Minimal 25 lokasi dan 40 jalur.

Tree:

- Menggunakan Trie untuk autocomplete nama lokasi, atau AVL Tree untuk menyimpan lokasi berdasarkan kode/nama.

Algoritma wajib:

- Dijkstra untuk mencari rute terbaik.
- BFS atau DFS untuk mengecek keterhubungan antar lokasi.

Fitur wajib:

1. Cari lokasi berdasarkan prefix.
2. Cari rute dari lokasi A ke lokasi B.
3. Tampilkan total jarak/waktu.
4. Simulasikan penutupan salah satu jalan.
5. Bandingkan rute sebelum dan sesudah perubahan.

Aspek HOTS:

Rute terpendek belum tentu rute terbaik. Kelompok harus menambahkan minimal satu faktor lain seperti keramaian, keamanan, akses difabel, atau jalan teduh.

Opsi 2 — Emergency Evacuation System

Membuat sistem rekomendasi jalur evakuasi dari ruangan atau gedung menuju titik aman.

Graph:

- Node merepresentasikan ruangan, koridor, tangga, pintu keluar, dan titik kumpul.
- Edge merepresentasikan jalur yang dapat dilewati.
- Edge memiliki bobot berupa jarak, waktu, atau tingkat risiko.
- Minimal 25 node dan 40 edge.

Tree:

- Menggunakan Min-Heap untuk memilih jalur dengan risiko terendah, atau AVL Tree untuk menyimpan titik evakuasi berdasarkan tingkat keamanan.

Algoritma wajib:

- BFS untuk mencari jalur evakuasi tercepat pada graph tak berbobot.
- Dijkstra untuk mencari jalur dengan risiko/biaya minimum.

Fitur wajib:

1. Input lokasi awal.
2. Cari jalur menuju titik aman.
3. Tampilkan jalur dan total risiko.
4. Simulasikan jalur tertutup.
5. Berikan alternatif jalur.

Aspek HOTS:

Kelompok harus menjelaskan bagaimana sistem mengambil keputusan jika jalur tercepat memiliki risiko tinggi, sedangkan jalur aman membutuhkan waktu lebih lama.

Opsi 3 — Food Delivery Route Optimizer

Membuat sistem sederhana untuk menentukan urutan pengantaran makanan berdasarkan lokasi, jarak, dan deadline.

Graph:

- Node merepresentasikan restoran, pelanggan, dan titik jalan.
- Edge merepresentasikan jalan antar lokasi.
- Edge memiliki bobot berupa jarak atau waktu tempuh.

- Minimal 25 node dan 40 edge.

Tree:

- Menggunakan AVL Tree untuk menyimpan order berdasarkan deadline, atau Heap untuk memilih order dengan prioritas tertinggi.

Algoritma wajib:

- Dijkstra untuk mencari jarak antar lokasi.
- BFS atau DFS untuk validasi keterhubungan area.

Fitur wajib:

1. Tambah order baru.
2. Cari order berdasarkan ID atau nama pelanggan.
3. Tentukan urutan pengantaran.
4. Hitung total jarak/waktu.
5. Simulasikan order mendadak dengan prioritas tinggi.

Aspek HOTS:

Kelompok harus menjelaskan trade-off antara mengantar berdasarkan jarak terdekat, deadline tercepat, atau prioritas pelanggan.

Opsi 4 — Library Knowledge Navigator

Membuat sistem pencarian buku/topik dan rekomendasi urutan belajar berdasarkan hubungan prasyarat antar materi.

Graph:

- Node merepresentasikan buku, bab, atau topik.
- Edge merepresentasikan hubungan prasyarat atau referensi.
- Graph wajib directed.
- Minimal 25 node dan 40 edge.

Tree:

- Menggunakan Trie untuk pencarian judul/topik, atau AVL Tree untuk menyimpan buku berdasarkan kode/topik.

Algoritma wajib:

- DFS untuk menelusuri hubungan topik.
- Topological Sort untuk memberi rekomendasi urutan belajar.

- Cycle Detection untuk mendeteksi prasyarat yang bermasalah.

Fitur wajib:

1. Search topik berdasarkan kata kunci/prefix.
2. Tampilkan daftar prasyarat sebuah topik.
3. Buat rekomendasi urutan belajar.
4. Deteksi siklus prasyarat.
5. Tampilkan topik yang tidak terhubung.

Aspek HOTS:

Kelompok harus menjelaskan apa yang terjadi jika terdapat siklus prasyarat, misalnya Topik A membutuhkan B, B membutuhkan C, dan C membutuhkan A.

Ops 5 — Public Transport Planner

Membuat sistem rekomendasi rute transportasi umum berdasarkan halte, stasiun, waktu tempuh, biaya, dan jumlah transit.

Graph:

- Node merepresentasikan halte, stasiun, atau terminal.
- Edge merepresentasikan rute transportasi.
- Edge memiliki minimal dua bobot, misalnya waktu dan biaya.
- Minimal 25 node dan 40 edge.

Tree:

- Menggunakan Heap untuk memilih kandidat rute terbaik, atau Trie untuk pencarian nama halte/stasiun.

Algoritma wajib:

- Dijkstra untuk rute berbobot.
- BFS untuk mencari rute dengan jumlah transit minimum.

Fitur wajib:

1. Cari halte/stasiun berdasarkan nama.
2. Cari rute tercepat.
3. Cari rute dengan transit paling sedikit.
4. Bandingkan dua kriteria rute.
5. Simulasikan satu rute yang tidak tersedia.

Aspek HOTS:

Kelompok harus menjelaskan mengapa rute tercepat, termurah, dan paling sedikit transit bisa

menghasilkan jawaban berbeda.

Opsi 6 — Game Map Pathfinding AI

Membuat sistem pencarian jalur untuk karakter game pada peta berbobot.

Graph:

- Node merepresentasikan tile, area, ruangan, atau checkpoint.
- Edge merepresentasikan jalur yang bisa dilewati.
- Edge memiliki bobot berdasarkan jenis terrain, misalnya tanah, air, pasir, lava, atau tembok.
- Minimal 25 node dan 40 edge.

Tree:

- Menggunakan Heap/Priority Queue untuk open list pada A*, atau AVL Tree untuk menyimpan checkpoint berdasarkan skor.

Algoritma wajib:

- A* untuk pathfinding.
- DFS atau BFS untuk eksplorasi map.

Fitur wajib:

1. Input titik awal dan tujuan.
2. Cari jalur menggunakan A*.
3. Tampilkan total cost.
4. Tampilkan area yang tidak dapat dilewati.
5. Ubah bobot terrain dan tampilkan perubahan rute.

Aspek HOTS:

Kelompok harus menjelaskan pengaruh heuristic terhadap kecepatan dan hasil pencarian jalur.

Opsi 7 — Hospital Patient Routing

Membuat sistem pengaturan alur pasien di rumah sakit berdasarkan poli, ruangan, prioritas, dan jalur layanan.

Graph:

- Node merepresentasikan poli, lab, apotek, kasir, ruang tunggu, atau IGD.

- Edge merepresentasikan alur perpindahan pasien.
- Edge memiliki bobot berupa waktu tempuh atau waktu tunggu.
- Minimal 25 node dan 40 edge.

Tree:

- Menggunakan Max-Heap untuk antrian pasien prioritas tinggi, atau AVL Tree untuk menyimpan pasien berdasarkan nomor antrean.

Algoritma wajib:

- BFS untuk mencari alur layanan.
- Dijkstra untuk jalur dengan waktu minimum.

Fitur wajib:

1. Tambah pasien.
2. Tentukan prioritas pasien.
3. Cari alur layanan pasien.
4. Tampilkan estimasi waktu.
5. Simulasikan pasien darurat masuk ke sistem.

Aspek HOTS:

Kelompok harus menjelaskan bagaimana sistem menyeimbangkan keadilan antrean dan prioritas pasien darurat.

Opsi 8 — Disaster Relief Distribution Network

Membuat sistem distribusi bantuan dari gudang ke posko bencana.

Graph:

- Node merepresentasikan gudang, posko, kota, desa, atau titik distribusi.
- Edge merepresentasikan jalan distribusi.
- Edge memiliki bobot berupa jarak, biaya, risiko, atau kapasitas.
- Minimal 25 node dan 40 edge.

Tree:

- Menggunakan Min-Heap untuk memilih distribusi berbiaya rendah, atau AVL Tree untuk menyimpan posko berdasarkan tingkat kebutuhan.

Algoritma wajib:

- Dijkstra untuk mencari jalur distribusi terbaik.

- Minimum Spanning Tree untuk merancang jaringan distribusi minimum.

Fitur wajib:

1. Tambah data posko dan kebutuhan.
2. Cari jalur pengiriman bantuan.
3. Tentukan prioritas posko.
4. Hitung total biaya distribusi.
5. Simulasikan jalan rusak atau tidak dapat dilewati.

Aspek HOTS:

Kelompok harus menjelaskan kapan MST cocok digunakan dan kapan Dijkstra lebih tepat digunakan.

Ops 9 — Course Prerequisite Planner

Membuat sistem rekomendasi urutan pengambilan mata kuliah berdasarkan prasyarat.

Graph:

- Node merepresentasikan mata kuliah.
- Edge merepresentasikan hubungan prasyarat.
- Graph wajib directed.
- Minimal 25 mata kuliah dan 40 relasi prasyarat.

Tree:

- Menggunakan Trie untuk pencarian nama mata kuliah, atau AVL Tree untuk menyimpan mata kuliah berdasarkan kode.

Algoritma wajib:

- Topological Sort untuk menentukan urutan pengambilan.
- Cycle Detection untuk mendeteksi prasyarat yang tidak valid.
- DFS untuk menampilkan rantai prasyarat.

Fitur wajib:

1. Cari mata kuliah berdasarkan prefix/kode.
2. Tampilkan prasyarat langsung dan tidak langsung.
3. Buat rekomendasi urutan pengambilan mata kuliah.
4. Deteksi siklus prasyarat.
5. Simulasikan penambahan mata kuliah baru.

Aspek HOTS:

Kelompok harus menjelaskan bagaimana sistem menangani konflik kurikulum, misalnya ketika mata

kuliah A membutuhkan B, tetapi B juga membutuhkan A.

Ketentuan

Untuk memastikan project tidak hanya dibuat dari template AI, setiap kelompok wajib menyertakan:

1. Dataset Buatan Sendiri

Dataset harus dibuat sendiri oleh setiap kelompok. Dataset tidak boleh hanya mengambil dari contoh di internet tanpa modifikasi.

- Minimal 25 node/vertex.
- Minimal 40 edge/relasi.
- Minimal 5 atribut data selain ID dan nama.
- Minimal 3 skenario pengujian normal.
- Minimal 3 edge case.

2. Struktur Data

- Minimal 1 struktur Tree yang diimplementasikan sendiri atau dimodifikasi secara signifikan.
- Minimal 1 struktur Graph yang diimplementasikan sendiri.
- Minimal 2 algoritma Graph.
- Minimal 1 operasi Tree selain insert dan search.
- Minimal 1 fitur yang menggabungkan hasil Tree dan Graph.

3. Analisis Struktur Data

Setiap kelompok wajib menjelaskan kompleksitas waktu untuk minimal 4 operasi utama, misalnya:

1. Insert data ke Tree.
2. Search data pada Tree.
3. Traversal Graph.
4. Shortest Path / MST / Topological Sort.

Format analisis kompleksitas:

Operasi	Struktur/ Algoritma	Kompleksitas	Alasan
Search lokasi	Trie	$O(L)$	L adalah panjang kata
Cari rute	Dijkstra	$O((V + E) \log V)$	Menggunakan priority queue

Operasi	Struktur/ Algoritma	Kompleksitas	Alasan
Traversal	BFS	$O(V + E)$	Mengunjungi semua vertex dan edge
Insert prioritas	Heap	$O(\log n)$	Perlu heapify-up

4. Design Decision Log

Kelompok wajib menuliskan minimal 5 keputusan desain.

Format:

No	Keputusan	Alternatif	Alasan Memilih	Risiko/ Kelemahan
1	Menggunakan adjacency list	Adjacency matrix	Lebih hemat memori untuk graph sparse	Cek edge langsung lebih lambat
2	Menggunakan Trie	HashMap	Butuh prefix search	Memori lebih besar

5. What-if Analysis

Kelompok wajib menjawab minimal 3 pertanyaan perubahan kondisi, misalnya:

1. Apa yang terjadi jika jumlah node naik dari 25 menjadi 10.000?
2. Apa yang terjadi jika edge tertentu dihapus?
3. Apa yang terjadi jika bobot edge berubah?
4. Apa yang terjadi jika input user tidak ditemukan?
5. Apa yang terjadi jika ada data duplikat?
6. Apa yang terjadi jika graph tidak terhubung?

6. Tracing Manual

Kelompok wajib menyertakan:

1. Satu tracing proses Tree.
2. Satu tracing proses Graph.
3. Satu tracing edge case.

7. Live Challenge Saat Presentasi

Saat presentasi, dosen/asisten dapat meminta kelompok melakukan salah satu perubahan berikut:

1. Tambahkan node baru.
2. Hapus edge tertentu.
3. Ubah bobot edge.
4. Cari data dengan prefix tertentu.
5. Jalankan ulang algoritma dari source ke destination baru.
6. Jelaskan mengapa output berubah.

Deliverables

Setiap kelompok wajib mengumpulkan:

1. Source Code Java

Berisi kode program lengkap yang dapat dijalankan.

Struktur folder disarankan:

FP_Strukdat_KelompokXX/

- src/
 - o Main.java
 - o graph/
 - o tree/
 - o model/
- data/
 - o dataset.csv atau dataset.txt
- docs/
 - o laporan.pdf
 - o tracing.pdf
- README.md

2. Laporan PDF

Laporan minimal berisi:

1. Judul project.

2. Nama anggota kelompok.
3. Deskripsi masalah.
4. Dataset.
5. Struktur Graph yang digunakan.
6. Struktur Tree yang digunakan.
7. Algoritma yang digunakan.
8. Design decision log.
9. Tracing manual.
10. Screenshot hasil program.
11. Analisis kompleksitas.
12. What-if analysis.
13. Kesimpulan.

3. Presentasi dan Demo

Durasi presentasi: 10–15 menit per kelompok.

Struktur presentasi:

1. 2 menit — Deskripsi masalah dan dataset.
2. 3 menit — Desain Tree dan Graph.
3. 3 menit — Demo fitur utama.
4. 2 menit — Kompleksitas dan edge case.
5. 2 menit — Q&A dan live challenge.

Rubrik Penilaian

Komponen	Bobot
Implementasi Tree, Graph, dan Algoritma	30%
Kesesuaian Solusi dengan Topik dan Requirement	20%
Kreativitas, HOTS, dan Keunikan Dataset	20%

Analisis Kompleksitas, Edge Case, dan What-if Analysis	15%
Demo, Kemampuan Menjelaskan, Kerapian Kode, dan Dokumentasi	15%
Total	100%

Penjelasan Singkat

1. Implementasi Tree, Graph, dan Algoritma — 30%

Menilai apakah struktur data Tree dan Graph benar-benar digunakan, algoritma berjalan dengan benar, dan fitur utama dapat dijalankan.

2. Kesesuaian Solusi dengan Topik dan Requirement — 20%

Menilai apakah project sesuai dengan opsi topik yang dipilih dan memenuhi minimum requirement.

3. Kreativitas, HOTS, dan Keunikan Dataset — 20%

Menilai keunikan ide, dataset buatan sendiri, serta kemampuan kelompok dalam membuat solusi yang tidak sekadar template.

4. Analisis Kompleksitas, Edge Case, dan What-if Analysis — 15%

Menilai penjelasan kompleksitas waktu, penanganan kasus khusus, dan analisis ketika kondisi berubah.

5. Demo, Kemampuan Menjelaskan, Kerapian Kode, dan Dokumentasi — 15%

Menilai kelancaran demo, pemahaman anggota kelompok, kerapian struktur kode, dan kelengkapan laporan/dokumentasi.

Catatan Penting

1. Mahasiswa bebas memilih topik, tetapi hanya dari 9 opsi yang tersedia. Topik di luar daftar tidak diperbolehkan.
2. Jika ada lebih dari satu kelompok yang memilih opsi yang sama, dataset, constraint, dan skenario masalah harus berbeda. (ada resiko kemungkinan similaritas tinggi)
3. Semua anggota kelompok **harus memahami isi program**.
4. Boleh menggunakan bantuan AI, tapi tidak boleh **hanya mengandalkan hasil Generate AI tanpa pemahaman**.
5. **Deadline: Senin, 15 Juni 2026, 09.00**

Selamat bersenang-senang 😊